Fondements de l'Algorithmique et Typage des Variables

Thème principal : Algorithmique impérative, types de données, structures conditionnelles et itératives

Synthèse exécutive : Ce cours pose les fondations indispensables de l'algorithmique impérative en BUT 1 Informatique. Il formalise le passage d'un problème métier à sa résolution automatisée par la machine, en détaillant la déclaration typée des variables, l'évaluation rigoureuse des expressions booléennes (avec la notion critique d'évaluation court-circuit) et la maîtrise des structures itératives bornées et non bornées accompagnées de leurs invariants et variants de boucle.

Fondements de l'Algorithmique et Typage des Variables

Un algorithme est une suite ordonnée et non ambiguë d'instructions permettant de résoudre un problème en un nombre fini d'étapes. Avant toute manipulation, la machine doit allouer de l'espace pour stocker les données. Les types primitifs fondamentaux sont les entiers (Integer), les réels/flottants (Float), les booléens (Boolean) et les chaînes de caractères (String). Une variable se caractérise par son identificateur, son adresse physique en mémoire RAM, son type qui détermine le nombre d'octets alloués et les opérations autorisées, et enfin sa valeur actuelle. Une confusion fréquente chez les débutants consiste à confondre l'opérateur d'affectation (noté <- ou =) et le test d'égalité mathématique (noté ==). L'affectation évalue d'abord l'expression située à droite, puis écrit le résultat dans la case mémoire désignée à gauche.

- Algorithme : suite ordonnée et non ambiguë d'instructions finies
- Variable : emplacement mémoire nommé doté d'un type et d'une valeur
- Affectation : opération destructrice liant une valeur à une variable

Cas concret enseignant : *La métaphore de la boîte aux lettres étiquetée : une variable est une boîte aux lettres portant un nom précis sur laquelle est fixé un format strict (on ne peut pas faire entrer un colis volumineux dans une fente à lettre). Si on remet un nouveau courrier dedans, l'ancien est irrémédiablement écrasé et détruit.*

Prise de Décision : Expressions Booléennes et Alternatives

L'exécution séquentielle linéaire ne suffit pas pour résoudre des problèmes réels ; le programme doit pouvoir dévier selon l'état des données. La structure conditionnelle 'Si (condition) Alors ... SinonSi ... Sinon ... FinSi' oriente le flux d'instructions. Les conditions reposent sur des opérateurs relationnels (<, <=, >, >=, ==, !=) combinés à l'aide d'opérateurs logiques. Une attention particulière doit être portée à l'évaluation court-circuit : dans une expression A ET B, si A est faux, l'interpréteur n'évalue même pas B. Cette propriété est cruciale pour éviter des erreurs fatales d'exécution, notamment lors de vérifications de pointeurs nuls ou d'accès hors limites dans un tableau.

- Prédicat : expression dont l'évaluation produit Vrai ou Faux
- Opérateurs logiques : ET, OU, NON régis par les lois de De Morgan
- Évaluation court-circuit : arrêt de l'évaluation dès que le résultat global est garanti

Cas concret enseignant : *Le tourniquet de métro à Anglet/Bayonne : pour franchir la barrière, la condition est (Pass_Valide ET Solde_Positif). Si le pass est périmé, le système refuse immédiatement sans même interroger la base bancaire distante, économisant du temps de calcul et évitant un plantage.*

Structures Itératives : Boucles Déterminées et Indéterminées

La répétition est la force motrice de l'informatique. On distingue deux grandes familles d'itérations : la boucle déterministe (Pour / For) et la boucle conditionnelle (Tant Que / While). La boucle 'Pour' initialise un compteur, vérifie la borne finale et applique un pas d'incrémentation à chaque tour. La boucle 'Tant Que' est indispensable lorsque le nombre d'itérations dépend d'un état dynamique (lecture d'un flux, saisie utilisateur, convergence

numérique). Le danger absolu d'une boucle 'Tant Que' est la boucle infinie, survenant lorsque la condition reste indéfiniment vraie. Pour garantir la terminaison d'un algorithme, l'étudiant doit toujours identifier un 'variant de boucle', c'est-à-dire une quantité entière positive qui décroît strictement à chaque itération jusqu'à atteindre zéro.

- Boucle Pour : boucle bornée dont le nombre d'itérations est connu à l'avance
- Boucle Tant Que : boucle non bornée avec test de condition préalable
- Variant de boucle : quantité entière positive qui décroît strictement pour assurer la terminaison

Cas concret enseignant : *Le distributeur automatique de billets : il utilise une boucle 'Tant Que' (tant que la somme demandée n'est pas atteinte, distribuer un billet adapté) plutôt qu'un 'Pour 3 billets', car selon le montant et les coupures disponibles, le nombre exact d'itérations est imprévisible à l'avance.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser une boucle 'Tant Que' si elle peut techniquement tout faire, y compris remplacer une boucle 'Pour' ?
Réponse du professeur : Techniquement, une boucle 'Tant Que' peut simuler n'importe quelle boucle 'Pour'. Cependant, la boucle 'Pour' est une structure hautement sémantique : elle indique explicitement au relecteur (et au compilateur) que le nombre d'itérations est connu et borné à l'avance. Utiliser un 'Pour' évite les oublis d'incrémentation du compteur et prévient par conception les risques de boucles infinies.

Question étudiant : Quelle est la différence concrète entre 'A OU B' et 'A OU_EXCLUSIF B' ?
Réponse du professeur : Le 'OU' inclusif (ou logique classique) est Vrai si A est vrai, si B est vrai, ou si les deux sont vrais en même temps. Le 'OU' exclusif (XOR) est Vrai si l'un des deux est vrai, mais il devient Faux si les deux sont vrais simultanément.

■ Exercice & Démonstration d'Application

Énoncé : Écrire l'algorithme complet permettant de calculer la moyenne des notes strictement positives saisies par l'utilisateur. La saisie s'arrête dès que l'utilisateur saisit la valeur sentinelle -1. Si aucune note valide n'a été entrée, l'algorithme doit afficher un message d'erreur approprié sans générer de division par zéro.

Résolution pas à pas :
1. Initialisation des accumulateurs : déclarer 'somme' à 0.0 (réel) et 'nb_notes' à 0 (entier). 2. Première lecture préalable (lecture anticipée) pour amorcer la condition de la boucle sentinelle : lire 'note'. 3. Structure de boucle : 'Tant Que note != -1 Faire'. 4. Vérification de validité interne : Si 'note > 0' Alors somme <- somme + note ; nb_notes <- nb_notes + 1 ; Sinon afficher 'Note invalide ignorée' ; FinSi. 5. Réitération de la lecture à l'intérieur de la boucle en dernière ligne : lire 'note'. 6. FinTantQue. 7. Traitement post-boucle : Si 'nb_notes > 0' Alors moyenne <- somme / nb_notes ; Afficher 'Moyenne : ', moyenne ; Sinon Afficher 'Aucune note valide saisie' ; FinSi.

Résultat : L'algorithme gère correctement la sentinelle d'arrêt (-1), évite le crash système de la division par zéro (si -1 est saisi dès le départ), et ignore les notes aberrantes négatives autres que la sentinelle.

■ Auto-Évaluation Flash d'Examen

Q : Que vaut l'expression booléenne : NON((5 > 2) ET (3 == 4)) ?

R : VRAI (car (5>2) est Vrai, (3==4) est Faux, leur ET vaut Faux, et la négation NON(Faux) donne Vrai).

Q : Quel est le risque majeur d'une mauvaise gestion de l'incrémentation dans une boucle 'Tant Que' ?

R : La boucle infinie, qui bloque l'exécution du programme et sature le processeur.

Q : Pourquoi la comparaison de deux nombres réels (flottants) avec l'opérateur '==' est-elle considérée comme une faute grave en programmation ?

R : En raison des imprécisions inhérentes à la représentation binaire des nombres à virgule flottante (norme IEEE 754), deux calculs mathématiquement égaux peuvent différer minimement au niveau du dernier bit.